

Experiment - 1

Aim – Write a Java program to demonstrate the concept of socket programming

Theory –

Socket programming is a method of connecting two nodes on a network to communicate with each other. This practical implements a chat application using TCP sockets in Java, demonstrating the client-server architecture where a server facilitates communication between two clients.

Key Components:

- **ServerSocket:** Listens for incoming client connections on port 5000
- **Socket:** Establishes the connection endpoint between client and server
- **BufferedReader:** Reads text from input streams efficiently
- **PrintWriter:** Sends formatted text to output streams with auto-flush
- **Multi-threading:** Enables simultaneous sending and receiving of messages

Source Code –

Client.java-

```
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);

            BufferedReader userInput = new BufferedReader(
                new InputStreamReader(System.in));

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));

            PrintWriter out = new PrintWriter(
                socket.getOutputStream(), true);

            System.out.println("Connected to server...");

            // Thread to read messages from server
            new Thread(() -> {
```

```

        try {
            String serverMsg;
            while ((serverMsg = in.readLine()) != null) {
                System.out.println(serverMsg);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }).start();

    String msg;
    while ((msg = userInput.readLine()) != null) {
        out.println(msg);
    }
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Server.java –

```

import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("Server started. Waiting for clients...");

            Socket client1 = serverSocket.accept();
            System.out.println("Client 1 connected.");

            Socket client2 = serverSocket.accept();
            System.out.println("Client 2 connected.");

            BufferedReader in1 = new BufferedReader(
                new InputStreamReader(client1.getInputStream()));
            PrintWriter out1 = new PrintWriter(client1.getOutputStream(),
true);

```

```

        BufferedReader in2 = new BufferedReader(
            new InputStreamReader(client2.getInputStream()));
        PrintWriter out2 = new PrintWriter(client2.getOutputStream(),
true);

        String msg;

        while (true) {
            if ((msg = in1.readLine()) != null) {
                System.out.println("Client 1: " + msg);
                out2.println("Client 1: " + msg);
            }
            if ((msg = in2.readLine()) != null) {
                System.out.println("Client 2: " + msg);
                out1.println("Client 2: " + msg);
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Output –

```

PS D:\Projects\ChatApp> javac Server.java
PS D:\Projects\ChatApp> java Server
Server started. Waiting for clients...
Client 1 connected.
Client 2 connected.
Client 1: Hello from Client 1
Client 2: Hi Client 1, how are you?
Client 1: I'm good! Thanks
Client 2: Great to hear that!

```

```

PS D:\Projects\ChatApp> javac Client.java
PS D:\Projects\ChatApp> java Client
Connected to server...
Connected to server. Client 2 is also connected.
Hello from Client 1
Client 2: Hi Client 1, how are you?
I'm good! Thanks
Client 2: Great to hear that!

```

Experiment - 2

Aim – Write a Java program to demonstrate the concept of applet programming

Theory –

An applet in Java is a small application designed to be embedded within a web page and executed in a web browser. It extends the Applet class and allows developers to create interactive and graphical user interfaces (GUIs) that can run within a browser's Java Virtual Machine (JVM).

Applets can respond to user inputs, handle events, and update their visuals dynamically. However, due to security concerns and the rise of modern web technologies, applets are deprecated and no longer supported in most browsers.

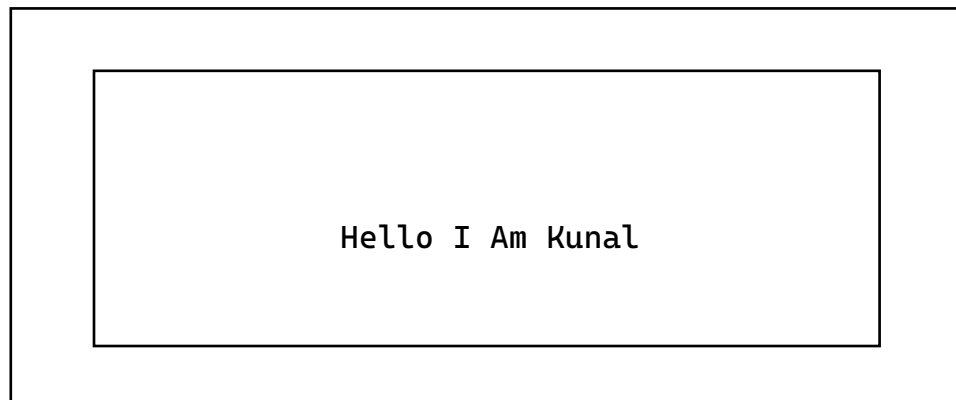
Source Code –

```
import java.applet.Applet;
import java.awt.Graphics;

public class AdvJava001_BasicApplet extends Applet {

    @Override
    public void paint(Graphics g) {
        g.drawRect(50, 50, 300, 100);
        g.drawString("Hello I Am Kunal", 100, 100);
    }
}
```

Output –



Experiment - 3

Aim – Write a Java program to demonstrate the concept of multi-threading

Theory –

Multithreading is a feature of Java that allows multiple threads to run concurrently within a program. A thread is the smallest unit of execution and is part of a process. Threads share the same memory but perform different tasks simultaneously, improving CPU utilization and performance.

Java provides two ways to create a thread:

1. By Extending the Thread Class

- A class extends the Thread class.
- The run() method is overridden to define the task.
- The thread starts using the start() method.

2. By Implementing the Runnable Interface

- A class implements the Runnable interface.
- The run() method contains the execution logic.
- A Thread object is created by passing the Runnable object.

Source Code –

```
public class MultithreadingDemo {

    // Method 1: Extend the Thread class
    static class ThreadClassExample extends Thread {
        public void run() {
            System.out.println("Thread-1 (Extending Thread) running");
            try {
                for (int i = 1; i <= 2; i++) {
                    System.out.println("Thread-1: " + i);
                    Thread.sleep(500); //
                }
            } catch (InterruptedException e) { }
        }
    }

    // Method 2: Implement the Runnable interface
    static class RunnableInterfaceExample implements Runnable {
        public void run() {
```

```

        System.out.println("Thread-2 (Implementing Runnable)
running");
        try {
            for (int i = 1; i <= 2; i++) {
                System.out.println("Thread-2: " + i);
                Thread.sleep(500);
            }
        } catch (InterruptedException e) { }
    }
}

public static void main(String[] args) {
    System.out.println("Main thread started.");

    ThreadClassExample t1 = new ThreadClassExample();
    t1.start(); //

    Thread t2 = new Thread(new RunnableInterfaceExample()); //
    t2.start();

    try {
        t1.join(); //
        t2.join();
    } catch (InterruptedException e) { }

    System.out.println("Main thread finished.");
}
}

```

Output –

```

Main thread started.
Thread-1 (Extending Thread) running
Thread-2 (Implementing Runnable) running
Thread-1: 1
Thread-2: 1
Thread-1: 2
Thread-2: 2
Main thread finished.

```

Experiment - 4

Aim – Write a Java program to demonstrate the concept of applet

Theory –

A Java Applet is a Java program that runs inside a web browser. An Applet is embedded in an HTML file using <applet> or <objects> tags. Applets are used to make the website more dynamic and entertaining. Applets are executed in a sandbox for security, restricting access to local system resources.

Key Points:

- Applet Basics: Every applet is a child/subclass of the java.applet.Applet class.
- Not Standalone: Applets don't run on their own like regular Java programs. They need a web browser or a special tool called the applet viewer (which comes with Java).
- No main() Method: Applets don't start with main() method.
- Display Output: Applets don't use System.out.println() for displaying the output, instead they use graphics methods like drawString() from the AWT (Abstract Window Toolkit).

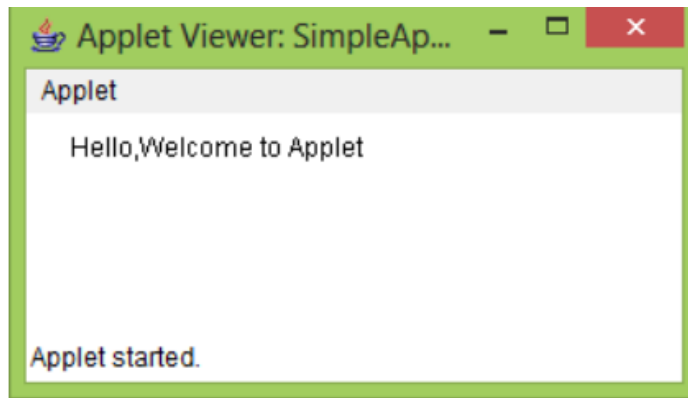
Source Code –

```
import java.awt.*;
import java.applet.*;
/*
<applet code="SimpleApplet" width=300 height=100>
</applet>
*/
public class SimpleApplet extends Applet
{
    String msg="";
    // Called first.
    public void init()
    {
        msg="Hello";
    }
    /*    Called second, after init().
```

```
Also called whenever the applet is restarted. */
public void start()
{
    msg=msg+",Welcome to Applet";
}

// whenever the applet must redraw its output, paint( ) is
called.
public void paint(Graphics g)
{
    g.drawString(msg,20,20);
}
}
```

Output –



Experiment – 5

Aim – Write a Java program to demonstrate the use of Java Beans.

Theory – A Java Bean is a reusable Java class that follows a set of conventions defined by Sun Microsystems. Beans are the building blocks of Java EE applications and are heavily used in JSP pages, Spring Framework, and GUI builders like NetBeans.

A class qualifies as a Java Bean if it satisfies three main rules:

1. It must have a public no-argument (default) constructor, so that tools and frameworks can instantiate it without knowing any constructor arguments.
2. All its fields (properties) must be private and accessible only through public getter and setter methods following the standard naming convention – getName() / setName() for a property called name.
3. The class must implement the java.io.Serializable interface so its state can be saved to disk or transferred across a network.

Source Code –

Student.java

```
import java.io.Serializable;

public class Student implements Serializable {

    private String name;
    private int    rollNo;
    private double marks;

    // No-arg constructor (mandatory for a Bean)
    public Student() { }

    public String getName()      { return name; }
    public void  setName(String n) { this.name = n; }

    public int  getRollNo()      { return rollNo; }
    public void setRollNo(int r)  { this.rollNo = r; }

    public double getMarks()      { return marks; }
    public void  setMarks(double m) { this.marks = m; }
}

// File: JavaBeanDemo.java (Main class)
public class JavaBeanDemo {
```

```
public static void main(String[] args) {  
  
    Student s1 = new Student();  
    s1.setName("Rahul Sharma");  
    s1.setRollNo(101);  
    s1.setMarks(92.5);  
  
    Student s2 = new Student();  
    s2.setName("Priya Patel");  
    s2.setRollNo(102);  
    s2.setMarks(87.0);  
  
    System.out.println("---- Student Bean Details ----");  
    System.out.println("Name   : " + s1.getName());  
    System.out.println("Roll No : " + s1.getRollNo());  
    System.out.println("Marks  : " + s1.getMarks());  
    System.out.println();  
    System.out.println("Name   : " + s2.getName());  
    System.out.println("Roll No : " + s2.getRollNo());  
    System.out.println("Marks  : " + s2.getMarks());  
    }  
}
```

Output –

```
user@ubuntu:~/JavaPrograms$ javac Student.java JavaBeanDemo.java  
user@ubuntu:~/JavaPrograms$ java JavaBeanDemo  
  
---- Student Bean Details ----  
Name       : Rahul Sharma  
Roll No    : 101  
Marks      : 92.5  
  
Name       : Priya Patel  
Roll No    : 102  
Marks      : 87.0  
  
user@ubuntu:~/JavaPrograms$
```

Experiment – 6

Aim – Write a Java program to insert data into a table using JSP.

Theory – JSP (JavaServer Pages) is a server-side technology that lets developers embed Java code directly inside HTML files. When a browser requests a JSP page, the web server (e.g. Apache Tomcat) compiles it into a Servlet, executes it, and sends the resulting HTML to the client.

JDBC (Java Database Connectivity) is the standard Java API for connecting to relational databases. It provides classes and interfaces in the java.sql package for executing SQL queries and retrieving results.

Core Concepts:

- JSP uses JDBC to connect to a database.
- Data is received from HTML form using request.getParameter().
- SQL INSERT query is used to store data in a table.
- Output is displayed using out.println().

Source Code –

insert.html

```
<!DOCTYPE html>
<html>
<head><title>Insert Student Record</title></head>
<body>
  <h2>Student Registration</h2>
  <form action="insertData.jsp" method="post">
    Name : <input type="text" name="sname" /><br/><br/>
    Roll No: <input type="text" name="sroll" /><br/><br/>
    City : <input type="text" name="scity" /><br/><br/>
    <input type="submit" value="Insert Record"/>
  </form>
</body>
</html>
```

insertData.jsp

```
<%@ page import="java.sql.*" %>
<%
  String name = request.getParameter("sname");
  int roll = Integer.parseInt(request.getParameter("sroll"));
  String city = request.getParameter("scity");

  Connection con = null;
  PreparedStatement ps = null;
```

```

try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    con = DriverManager.getConnection("jdbc:mysql://localhost:3306/college", "root", "root123");

    ps = con.prepareStatement("INSERT INTO students(name, roll, city) VALUES(?,?,?)");
    ps.setString(1, name);
    ps.setInt(2, roll);
    ps.setString(3, city);

    int rows = ps.executeUpdate();
    if (rows > 0) {
        out.println("<h3>Record inserted successfully!</h3>");
        out.println("<p>Name: " + name + ", Roll: " + roll
            + ", City: " + city + "</p>");
    }
} catch (Exception e) {
    out.println("Error: " + e.getMessage());
} finally {
    if (ps != null) ps.close();
    if (con != null) con.close();
}
%>

```

Output –

Student Registration

Name :

Roll No:

City :

Record inserted successfully!

```

+---+-----+---+-----+
| id | name          | roll | city    |
+---+-----+---+-----+
| 1  | Rahul Sharma  | 101  | Nagpur  |
+---+-----+---+-----+

```

1 row in set (0.01 sec)

Experiment – 7

Aim – Write a JSP program to implement form data validation.

Theory –

Form validation is the process of verifying that the data entered by the user in an HTML form is correct, complete, and in the expected format before it is processed or stored. Validation can be performed either on the client side (using JavaScript) or on the server side (using Java/JSP). Server-side validation is always mandatory because a user can bypass client-side checks by disabling JavaScript.

In JSP, the server receives form data as request parameters (plain strings). The JSP code then inspects each parameter and builds an error list. If no errors are found the data is processed; otherwise the errors are shown back to the user.

Core Concepts:

- Data is received using `request.getParameter()`.
- Validation checks include empty fields, length, format, etc.
- Errors are displayed using `out.println()`.
- Ensures only valid data is processed further.

Source Code –

registrationForm.html

```
<!DOCTYPE html>
<html>
<head><title>Registration Form</title></head>
<body>
  <h2>User Registration</h2>
  <form action="validate.jsp" method="post">
    Name : <input type="text" name="uname" /><br/><br/>
    Email : <input type="text" name="email" /><br/><br/>
    Age : <input type="text" name="age" /><br/><br/>
    <input type="submit" value="Register"/>
  </form>
</body>
</html>
```

validate.jsp

```
<%@ page import="java.util.*" %>
<%
  String name = request.getParameter("uname");
  String email = request.getParameter("email");
```

```

String ageStr = request.getParameter("age");
List<String> errors = new ArrayList<>();

if (name == null || name.trim().isEmpty())
    errors.add("Name is required.");

if (email == null ||
    !email.matches("^[\\w.-]+@[\\w.-]+\\.\\w+$"))
    errors.add("Invalid email address.");

try {
    int age = Integer.parseInt(ageStr);
    if (age < 18 || age > 100)
        errors.add("Age must be between 18 and 100.");
} catch (NumberFormatException e) {
    errors.add("Age must be a valid number.");
}

if (errors.isEmpty()) {
    out.println("<h3>Registration Successful! Welcome, " + name + "</h3>");
} else {
    out.println("<h3>Validation Errors:</h3><ul>");
    for (String err : errors)
        out.println("<li>" + err + "</li>");
    out.println("</ul>");
    out.println("<a href='registrationForm.html'>Go Back</a>");
}
%>

```

Output –

Student Registration Form

Name:

Age:

Email:

Registration Successful! Welcome, Rahul Sharma

Experiment – 8

Aim – Write a Java program to show user validation using Servlet.

Theory – A Java Servlet is a server-side component that handles HTTP requests and produces HTTP responses. Servlets are managed by a Servlet container (such as Apache Tomcat) and run inside the container's JVM. Unlike CGI scripts, Servlets are loaded into memory only once and can handle multiple requests concurrently using threads, making them far more efficient.

The Servlet lifecycle has three phases:

1. `init()` – called once when the Servlet is first loaded. Used for one-time initialisation like opening database connections.
2. `service()` – called for every request. The default implementation delegates to `doGet()` or `doPost()` based on the HTTP method.
3. `destroy()` – called once before the Servlet is unloaded. Used for cleanup such as closing connections.

Source Code –

login.html

```
<!DOCTYPE html>
<html>
<head><title>User Login</title></head>
<body>
  <h2>Login Page</h2>
  <form action="LoginServlet" method="post">
    Username : <input type="text" name="username" /><br/><br/>
    Password : <input type="password" name="password" /><br/><br/>
    <input type="submit" value="Login"/>
  </form>
</body>
</html>
```

LoginServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet
{
    public void doPost(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException
    {
        response.setContentType("text/html");
```

```
        PrintWriter out = response.getWriter();
        String user = request.getParameter("username");
        String pass = request.getParameter("password");
        if(user.equals("admin") && pass.equals("1234"))
        {
            out.println("Login Successful");
        }
        else
        {
            out.println("Invalid Username or Password");
        }
    }
}
```

Output –

Login Page

Username :

Password :

Login

Login Successful

Experiment – 9

Aim – Write a program to set cookie information using Servlet.

Theory –

HTTP is a stateless protocol – the server treats every request independently and has no memory of previous ones. Cookies were introduced to overcome this limitation. A cookie is a small text-based key-value pair that the server sends to the browser in an HTTP response header (Set-Cookie). The browser stores the cookie and automatically includes it in every subsequent request to the same domain (Cookie header), enabling the server to identify returning users.

Cookies have a 4 KB size limit per cookie, and browsers typically allow 20-50 cookies per domain. For storing larger amounts of data, server-side sessions (HttpSession) backed by a session ID cookie are used instead. Sensitive data should never be stored directly in a cookie; if necessary, the cookie should be encrypted and marked with the HttpOnly and Secure flags.

Source Code –

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SetCookieServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        // Create two cookies
        Cookie userCookie = new Cookie("username", "RahulSharma");
        Cookie roleCookie = new Cookie("role", "student");

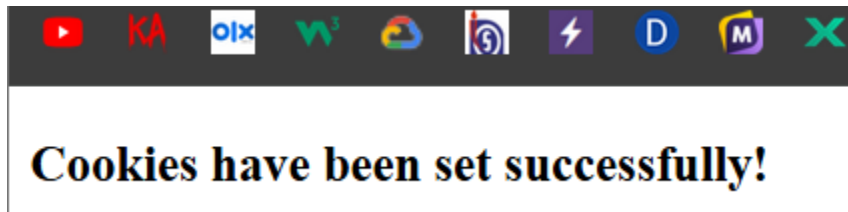
        // Set expiry to 1 hour (3600 seconds)
        userCookie.setMaxAge(3600);
        roleCookie.setMaxAge(3600);

        // Make cookies available to the whole app
        userCookie.setPath("/");
        roleCookie.setPath("/");

        // Add cookies to response
```

```
response.addCookie(userCookie);  
response.addCookie(roleCookie);  
  
response.setContentType("text/html");  
PrintWriter out = response.getWriter();  
out.println("<h3>Cookies have been set successfully!</h3>");  
}  
}
```

Output –



Experiment – 10

Aim – Develop a small web program using Servlets, JSPs with Database connectivity.

Theory –

process data, and generate dynamic responses on a web server such as Apache Tomcat.

A Servlet is a Java class that runs on the server and processes requests from the client. It acts as a controller in a web application by receiving data from the user, performing processing or business logic, and interacting with the database if required. Servlets are capable of handling HTTP requests using methods such as doGet() and doPost().

A Java Server Page (JSP) is used to design the presentation layer of the web application. JSP allows developers to combine HTML with Java code to create dynamic web pages. It simplifies web development by separating the user interface from the business logic

Core Concepts:

- Servlet processes request and controls flow
- JSP displays output (view layer)
- JDBC connects to database and executes queries
- Follows MVC (Model-View-Controller) architecture

Source Code –

index.jsp

```
<%@ page contentType="text/html" %>
<html>
<body>
<h2>Student Registration</h2>

<form action="RegisterServlet" method="post">
Name: <input type="text" name="name"><br><br>
Roll: <input type="text" name="roll"><br><br>
Branch: <input type="text" name="branch"><br><br>

<input type="submit" value="Register">
</form>

<br>
<a href="ViewServlet">View Students</a>

</body>
</html>
```

RegisterServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class RegisterServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req,
                          HttpServletResponse res)
        throws ServletException, IOException {

        String name = req.getParameter("name");
        String roll = req.getParameter("roll");
        String branch = req.getParameter("branch");

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root", "root123"
            );

            Statement st = con.createStatement();

            st.executeUpdate("INSERT INTO students VALUES("
                + name + "," + roll + "," + branch + ")");

            out.println("<h3>Student Registered Successfully!</h3>");

            con.close();

        } catch (Exception e) {
            out.println("Error: " + e);
        }

        out.println("<br><a href='index.jsp'>Back</a>");
    }
}
```

ViewServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class ViewServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req,
                          HttpServletResponse res)
        throws ServletException, IOException {

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        out.println("<h2>Student List</h2>");
        out.println("<table border=1>");
        out.println("<tr><th>Name</th><th>Roll</th><th>Branch</th></tr>");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root", "root123"
            );

            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM students");

            while (rs.next()) {
                out.println("<tr>");
                out.println("<td>" + rs.getString(1) + "</td>");
                out.println("<td>" + rs.getInt(2) + "</td>");
                out.println("<td>" + rs.getString(3) + "</td>");
                out.println("</tr>");
            }

            out.println("</table>");
            con.close();

        } catch (Exception e) {
            out.println("Error: " + e);
        }
    }
}
```

```
    }  
    out.println("<br><a href='index.jsp'>Back</a>");  
  }  
}
```

SQL Schema

```
CREATE TABLE students (  
  name VARCHAR(50),  
  roll INT,  
  branch VARCHAR(20)  
);
```

Output –

Student Registration

Name:

Roll:

Branch:

All Registered Students

Name	Roll	Branch
Rahul Sharma	101	CSE
Priya Patel	102	IT
Amit Kumar	103	ECE

[Back to Registration](#)